

last updated: 2026-01-30

author: 강민준(joonamin44@gmail.com)

docs 누적 방법

폴더 구조

파일 작성 예시

0. 공통 frontmatter

1. Context Map

2. docs/ai_context/

3. docs/adr/ADR-002

4. docs/specs

워크 플로우

docs 누적 방법

- 목적
 - AI에게 '기억'과 '의도'를 심어주어 보다 더 의도에 맞는 코드를 생성하기 위함
 - AI를 위한 context를 보강할 수 있도록 Single Source of Truth를 문서로 관리하기 위함
 - 항상 문서는 최신화 상태를 유지해야한다.

폴더 구조

```
1 project-root/
2 |— docs/
3 |   |— context_map.md      # 🗺️ (지도) 모든 문서의 진입점
4 |   |
5 |   |— ai_context/        # 🧱 (기반) 변하지 않는 사실들
6 |   |   |— tech_stack.md   # 사용 기술 및 버전
7 |   |   └— project_structure.md
8 |   |
9 |   |— conventions/       # 📏 (규칙) 코드 스타일, 린트 규칙
10 |   |   |— naming.md
11 |   |   └— code_style.md
12 |   |
13 |   |— adr/               # ⚖️ (판결) 아키텍처 의사결정 (Why)
14 |   |   |— 001-architecture.md
15 |   |   └— 002-state-mgmt.md
16 |   |
17 |   └— specs/             # 📝 (설계도) 구현해야 할 기능 명세 (What)
18 |       |— auth/          # (규모가 크면 폴더로 분류)
19 |       |   |— login.md
20 |       |   └— signup.md
21 |       └— payment.md
22 |       └— chat_system.md
```

- AI가 필요한 정보를 `0(1)`로 찾을 수 있도록 계층형으로 구조화하여 제공
- 모든 작업이 끝나면, `context_map` 파일은 필수적으로 업데이트 되어야한다.

- 자동화 스크립트 고민 중
- 기본적으로 모든 문서는 **frontmatter**를 작성한다
 - 일종의 메타데이터로 각각의 파일에 맞는 frontmatter를 작성한다. (각 디렉토리의 template.md 파일 참고)
 - context_map은 일종의 라우팅 역할을 하는 문서이므로 frontmatter를 작성하지 않는다.
- adr은 기술적 의사결정을 누적하는 폴더로, 0001-{제목}.md 로 작성한다.
 - 카운터는 시간의 흐름에 따라서 누적한다.
 - 이 결정이 앞으로의 모든 개발에 영향을 미치는 경우 작성한다.
 - 새로운 기술/라이브러리 도입:
 - 예: "Axios 대신 Fetch API만 쓰기로 결정함."
 - 예: "상태 관리를 Redux에서 Zustand로 교체함."
 - 구조적 패턴 변경:
 - 예: "지금부터 모든 컴포넌트는 Container-Presenter 패턴을 버리고 Hooks 패턴을 사용한다."
 - 예: "API 폴더 구조를 도메인별로 묶기로 변경한다."
 - 비기능적 요구사항의 합의 (Trade-off):
 - 예: "초기 로딩 속도를 위해 SEO를 일부 포기하고 CSR(Client Side Rendering)을 유지한다."
 - 되돌리기 힘든 결정 (One-way Door):
 - 예: "DB를 MongoDB에서 PostgreSQL로 마이그레이션 한다."
 - 단순한 버그 수정, 기존 규칙을 따라서 기능 추가하는 경우, 변수명 변경, 리팩토링, UI/CSS 디자인 변경인 경우는 ADR을 작성하지 않는다.

파일 작성 예시

0. 공통 frontmatter

```

1  ---
2  title: 문서의 공식 명칭.
3  type: 문서의 성격 (system | convention | adr | spec).
4  status: 현재 유효성 (active | accepted | deprecated | draft).
5  tags: 키워드를 배열로 정의
6  trigger_intent: (핵심) AI가 이 문서를 언제 읽어야 하는지 자연어로 정의.

```

1. Context Map

```

1  # 🗺️ Project Context Map
2  > **AI에게:** 이 파일은 프로젝트의 네비게이션 지도입니다. 사용자의 요청을 받으면, 가장 먼저 이
   파일을 읽고 **'어떤 상세 문서가 필요한지'** 판단하여 요청하세요.
3  > last updated: 2026-01-30 17:00
4  ## 1. Core Context (기반 지식)
5  > 프로젝트의 기술 스택, 구조, 핵심 원칙 등 변하지 않는 근간입니다.
6
7  | 문서 | 설명 | ⚡ Trigger Intent (이럴 때 참조하세요) |
8  | :---|:---|:---|
9  | \[Tech Stack\](./ai_context/tech_stack.md) | 언어, 프레임워크, 라이브러리 버전 명세
   | 패키지 설치, 버전 호환성 확인, 환경 설정 시 |
10 | \[Folder Structure\](./ai_context/structure.md) | 디렉토리 구조 및 역할 정의 |
   새 파일 생성 위치 결정, 모듈 찾기 어려울 때 |
11 | \[Naming Convention\](./conventions/naming.md) | 변수명, 파일명, 커밋 메시지 규칙
   | 코드 스타일링, 리팩토링, 변수 작명 시 |
12
13 ## 2. Key Architecture Decisions (ADR)
14 > 우리가 '왜' 기술적인 결정을 내렸는지에 대한 합의 사항입니다. (status: Accepted 만 포함)
15
16 | ID | 문서 | ⚡ Trigger Intent (이럴 때 참조하세요) |
17 | :---|:---|:---|
18 | 003 | \[State Management\](./adr/003-zustand.md) | **전역 상태(Store)** 추가/
   수정, 상태 관리 로직 설계 시 |
19 | 005 | \[Auth Flow\](./adr/005-auth.md) | **로그인/회원가입**, JWT 토큰 처리, 보안
   로직 수정 시 |
20 | 008 | \[React Query\](./adr/008-server-state.md) | **API 데이터 페칭**, 캐싱,
   비동기 데이터 처리 시 |
21
22 ## 3. Feature Specifications (도메인 로직)
23 > 비즈니스 로직과 기능 명세입니다. 기능 구현 시 가장 많이 참조됩니다.
24
25 | 문서 | 설명 | ⚡ Trigger Intent (이럴 때 참조하세요) |
26 | :---|:---|:---|
27 | \[Payment System\](./specs/payment.md) | 결제 프로세스 및 PG사 연동 가이드 | 결제
   오류 수정, 환불 로직, 정산 기능 구현 시 |
28 | \[User Profile\](./specs/profile.md) | 사용자 정보 수정 및 조회 로직 | 마이페이지,
   프로필 이미지 업로드, 닉네임 변경 시 |
29 | \[Chat System\](./specs/chat.md) | 실시간 채팅 및 소켓 로직 | 메시지 전송, 채팅방 생
   성, 알림 기능 구현 시 |

```

2. docs/ai_context/

```
1 ---
2 title: "기술 스택 및 버전 명세"
3 type: "system"
4 status: "active"
5 last_updated: "2026-01-30"
6 tags: ["tech", "versions", "libraries"]
7 trigger_intent:
8   - "새로운 라이브러리를 설치하려고 할 때"
9   - "패키지 버전 호환성을 확인할 때"
10  - "프로젝트 환경 설정을 할 때"
11 ---
12
13 # 🛠️ Tech Stack & Versions
14
15 ## Core
16 - **Language:** TypeScript 5.3 (Strict Mode)
17 - **Framework:** Next.js 14.1 (App Router)
```

3. docs/adr/ADR-002

```
1 ---
2 title: "ADR-002: 전역 상태 관리로 Zustand 채택"
3 type: "adr"
4 status: "accepted"
5 date: "2026-01-30"
6 tags: ["state", "store", "zustand", "redux"]
7 trigger_intent:
8   - "전역 상태(Store)를 새로 생성하거나 수정할 때"
9   - "컴포넌트 간 데이터 공유 로직을 짤 때"
10  - "Redux나 Context API 사용을 고려할 때(금지 확인)"
11 ---
12
13 # [ADR-002] 전역 상태 관리로 Zustand 채택
14 ## 1. Context & Problem
15 Redux의 높은 보일러플레이트와 복잡성으로 인해 개발 속도가 저하됨.
16 React Context API는 잦은 리렌더링 문제가 발생함.
17
18 ## 3. Decision
19 우리는 **Zustand**를 전역 상태 관리 라이브러리로 사용한다.
20
21 ## 4. 🤖 AI Guidelines (Instructions)
```

```

22 > AI는 상태 관리 코드를 작성할 때 다음 규칙을 준수해야 한다.
23
24 1. **Store 생성:** `create` 함수를 사용하여 `src/stores/` 폴더에 작성한다.
25 2. **Naming:** Store hook의 이름은 반드시 `use`로 시작하고 `Store`로 끝난다. (ex:
    `useAuthStore`)
26 3. **Selector Pattern:** 리렌더링 최적화를 위해 전체 state를 가져오지 말고 필요한 값만
    가져온다.
27     - ❌ `const { user } = useAuthStore()`
28     - ✅ `const user = useAuthStore((state) => state.user)`

```

4. docs/specs

```

1  ---
2  title: "로그인 기능 명세 (Login Spec)"
3  type: "spec"
4  status: "active"
5  related_components: ["LoginForm", "useAuthStore"]
6  tags: ["auth", "login", "jwt", "security"]
7  trigger_intent:
8    - "로그인 로직(API 호출, 유효성 검사)을 구현할 때"
9    - "JWT 토큰 저장 및 관리 방식을 수정할 때"
10   - "로그인 실패/성공 시 리다이렉트 처리를 할 때"
11  ---
12
13  # 🗝 로그인 기능 명세
14
15  ## 1. Overview
16  사용자가 이메일과 비밀번호를 사용하여 시스템에 접속하고, JWT 토큰을 발급받는 과정.
17
18  ## 2. 입력 데이터 (Input Fields)
19  | 필드명 | 타입 | 필수여부 | 유효성 검사 규칙 (Validation) |
20  | :--- | :--- | :--- | :--- |
21  | Email | string | Yes | 이메일 형식 준수 (@ 포함) |
22  | Password | string | Yes | 최소 8자 이상, 특수문자 1개 이상 포함 |
23
24  ## 3. 동작 로직 (Business Logic)
25  // 자연어로 기술하기 모호한 경우 mermaid 다이어그램으로 그려보자!
26  1. **제출(Submit) 클릭 시:**
27     - 클라이언트 측 유효성 검사를 먼저 수행한다.
28     - 실패 시 input 하단에 빨간색 에러 메시지를 노출한다.
29  2. **API 요청:**
30     - `POST /api/v1/auth/login` 엔드포인트를 호출한다.
31  3. **응답 처리:**
32     - 성공 (200) 시

```

```

32 - **성공 (200):**
33   - Access Token을 LocalStorage에 저장하지 않고, **HttpOnly Cookie**에 저장한
다. (보안 중요!)
34   - 메인 페이지(`/dashboard`)로 이동시킨다.
35 - **실패 (401 - 비밀번호 틀림):**
36   - "아이디 또는 비밀번호가 일치하지 않습니다" 토스트 메시지를 띄운다.
37 - **실패 (403 - 이메일 미인증):**
38   - 인증 메일 재발송 모달을 띄운다.
39
40 ## 4. 예외 처리 (Edge Cases)
41 - **네트워크 오류:** "네트워크 연결을 확인해주세요" 메시지 표시.
42 - **이미 로그인된 상태:** 로그인 페이지 접근 시 대시보드로 리다이렉트.

```

워크 플로우

1. Plan (계획 & 탐색):

- 개발자: "회원가입 폼에 전화번호 인증 기능을 추가해줘."
- AI: (`context_map.md` 확인) "알겠습니다. 인증 관련이므로 `docs/adr/005-auth-flow.md` 와 현재의 `schema.prisma` 를 읽고 시작하겠습니다."

2. Code (구현):

- AI: 문서의 제약 조건(ADR, Convention)을 준수하여 코드를 생성.

3. Update (문서 동기화 - 가장 중요 ★):

- 기능 구현이 끝나면, **AI에게 문서를 업데이트하라고 시킨다.**
- 개발자: "방금 구현하면서 변경된 DB 스키마 내용을 `docs/db/schema-design.md` 에 반영하고, 새로운 결정 사항이 있다면 `docs/adr/` 에 초안을 써줘."





